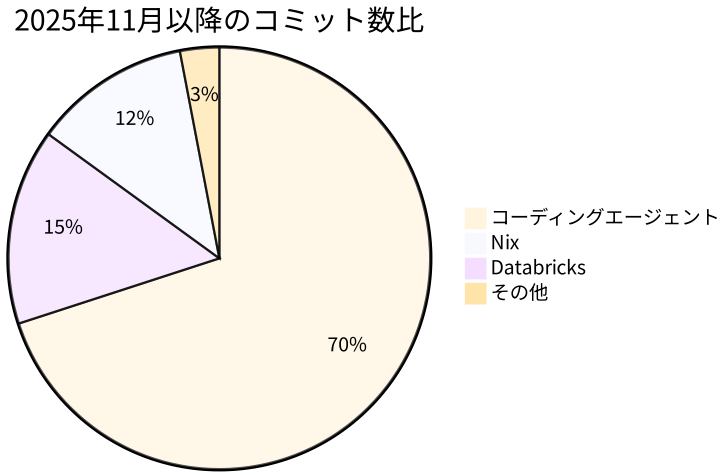
Databricksで堅牢なコードを書く上での課題とその解決

uma-chan

2026-03-06

1. 自己紹介

- 名前：uma-chan
- 職種：データエンジニア / MLOpsエンジニア
- 最近気になるもの
  - コーディングエージェント
  - Nix
  - Databricks



1.1. 登壇までの経緯

堅牢.py #1 (2025-11-15) で「堅牢な Python を書く」話を聞いているときの脆弱な私

堅牢.pyのおかげで **「Databricks でもローカル開発やテストをちゃんとしたい」** という課題意識が高まって jupyter-databricks-kernel を作ったので今日はその話をします！

2. jupyter-databricks-kernel

2.1. Databricks とは

- AWS / Azure / Google Cloud 上で動くマネージド Spark / Python 分析基盤
- Jupyter Notebook でコードを書いてクラスター上で実行する
- クラスター = 裏側はクラウド計算リソース (弊社は AWS)
- データ処理・ML 用途が多い

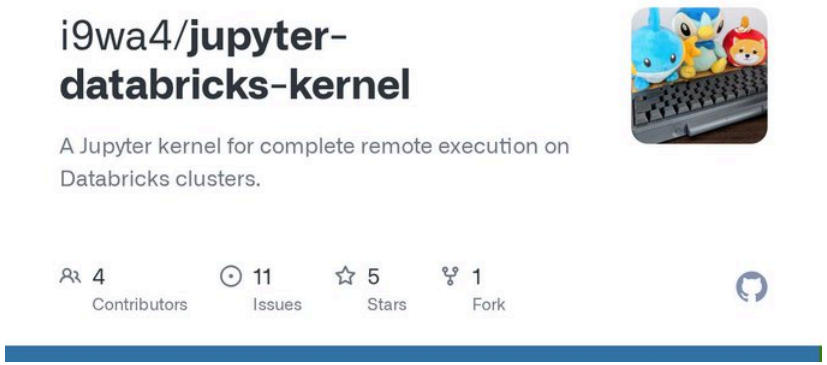
2.2. 解決したい課題

Databricks で堅牢なコードを書くにはローカル開発環境が必要。しかし…

課題	詳細
ローカル開発との乖離	VS Codeが使いにくい、Git連携が煩雑
AIツールからの実行困難	Claude CodeなどがDatabricksコードを自律実行できない
VS Code拡張の制約	Spark操作はリモート — プリインストールライブラリがローカルで使えない

2.3. jupyter-databricks-kernel とは

ローカルの Jupyter Notebook を Databricks クラスターで直接を実行させるためのツールを作りました



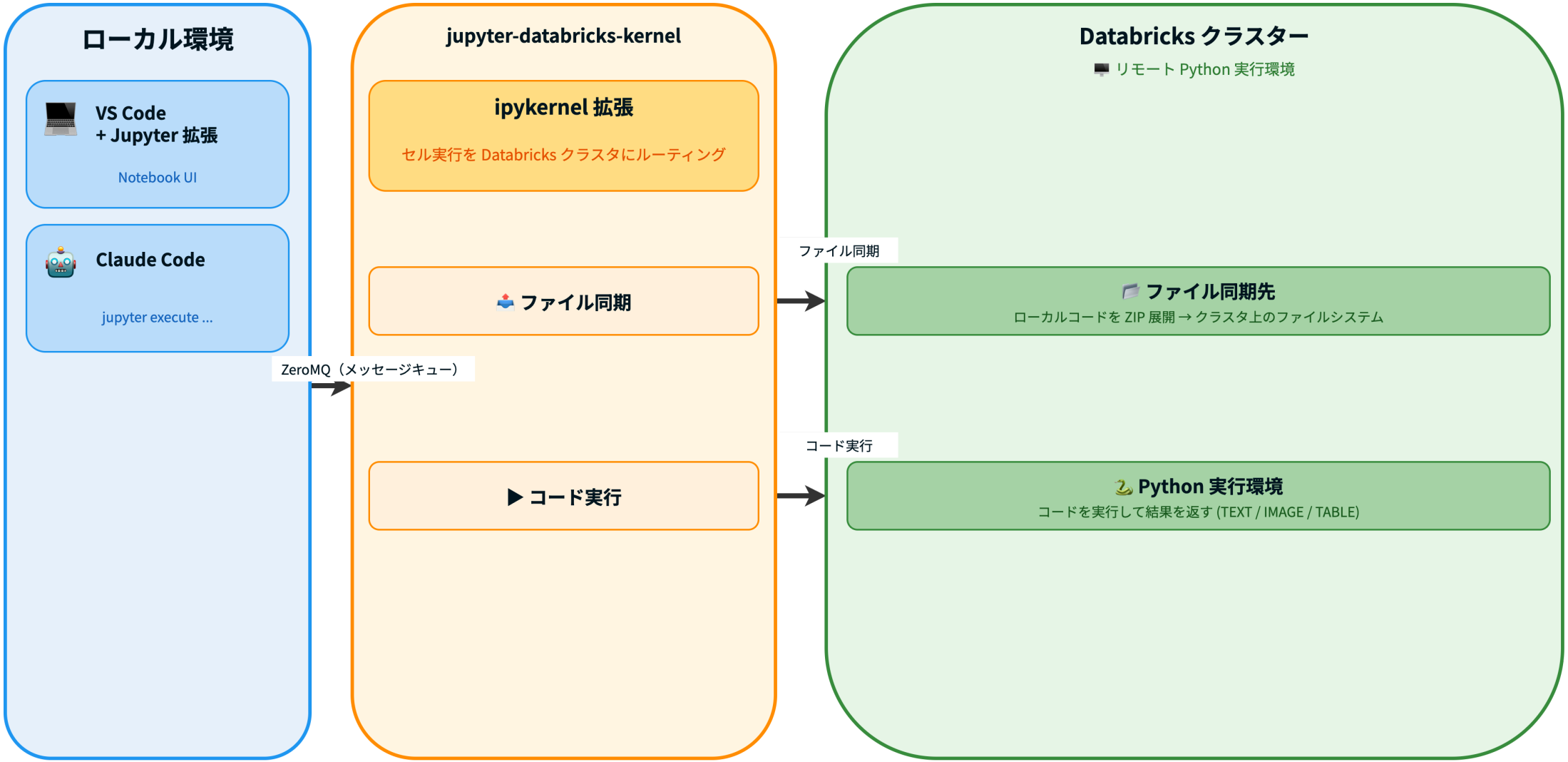
jupyter-databricks-kernel README

https://github.com/i9wa4/jupyter-databricks-kernel

2.4. Jupyterカーネルとは

- Jupyter カーネル = セルを実行するプロセス
- VS Code や JupyterLab からセルを実行すると、裏側でカーネルプロセスが動いている
- 普段は Python (ipykernel) が動いている
- クライアント↔カーネル間はZeroMQ（非同期メッセージキュー）で通信する
- カーネルは差し替え可能 — jupyter-databricks-kernel はその「差し替え」
- セル実行を横取りして、Databricks クラスターに投げる

2.5. アーキテクチャ



ローカルの Jupyter クライアントから Databricks クラスタで完全リモート実行 — コードはすべてクラスタ上で動く

2.6. 完全リモート実行とは

Databricks Connect との違い

項目	Databricks Connect	jupyter-databricks-kernel
Spark 計算	クラスター	クラスター
`.toPandas()` 結果	ローカルメモリに collect	クラスターのメモリ上に存在
通常の Python 変数	ローカル	クラスター上

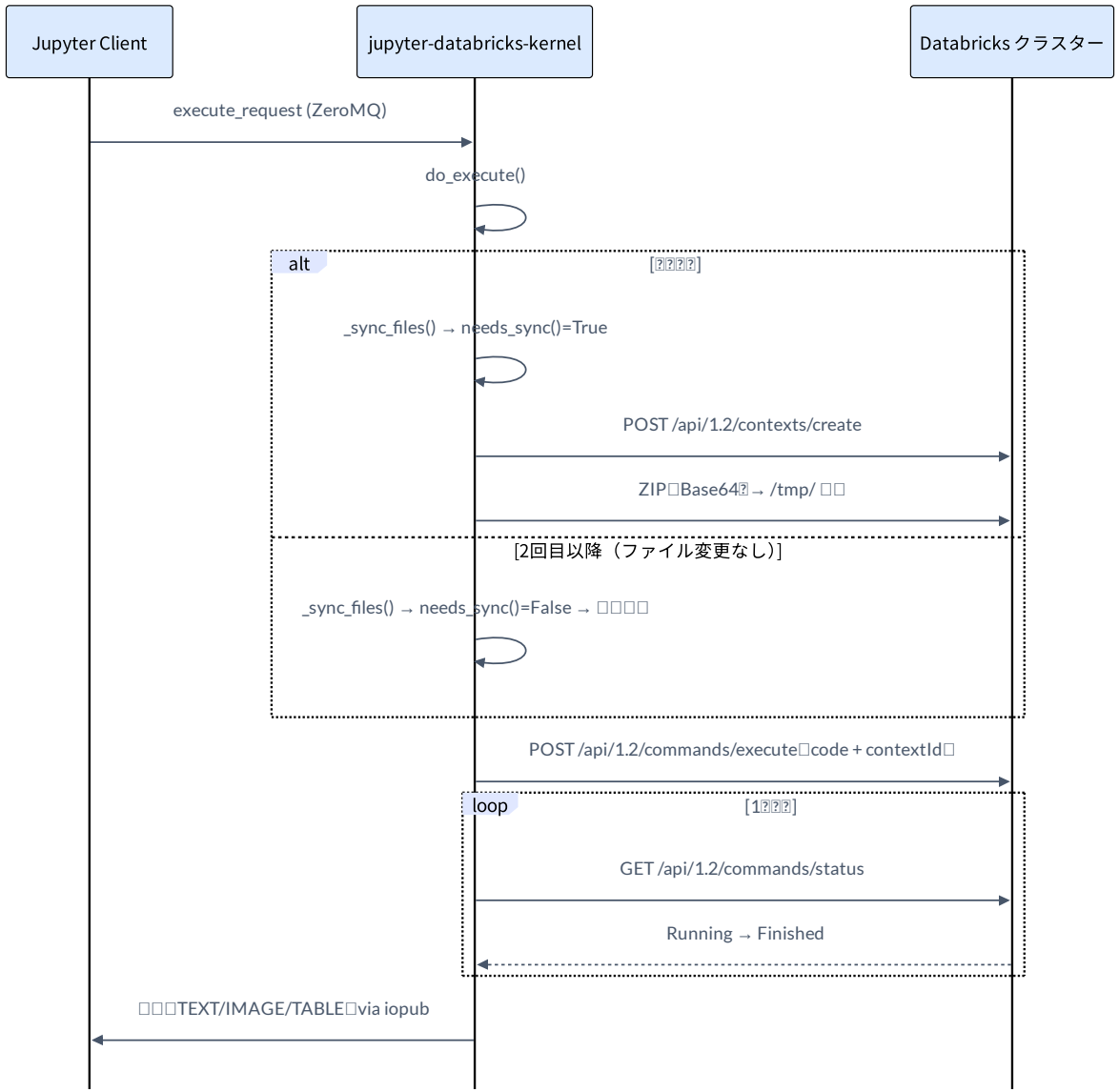
例: 大量データをローカルに取り込む場合

```
# 100万行のSparkDataFrameをpandasに変換
df = spark.sql("SELECT * FROM large_table LIMIT 1000000").toPandas()

# Databricks Connect の場合:
#   → 100万行がローカルマシンのメモリに転送される
#   → ローカルのメモリ不足 / 転送に時間がかかる

# jupyter-databricks-kernel の場合:
#   → df はクラスター上のメモリに存在
#   → ローカルには結果の表示のみ返る
```

2.7. セル実行のステップ



2.8. 使い方

Step 1: インストール

```
pip install jupyter-databricks-kernel
python -m jupyter_databricks_kernel.install
```

Step 2: 認証設定

```
databricks auth login --configure-cluster
```

Step 3: VS CodeでJupyter拡張を開き、カーネルに “Databricks” を選択

2.9. AIツールとの連携

- CLI実行でAIによる自律実行が可能:

```
jupyter execute notebook.ipynb --kernel_name=databricks --inplace
```

- Claude Code / Codex CLIからDatabricksコードを**自律的に実行・反復**できる
- CI/CDパイプラインへの組み込みも可能
- ファイル同期: 実行前にローカルファイルをクラスターへ自動アップロード (`.gitignore` 対応、ディレクトリ構造を保持)

2.10. これで堅牢な Python が書けるようになる理由

- **pytest ローカル実行**: クラスターコードをローカルでテスト → CI に組み込める
- **CI/CD 組み込み**: `jupyter execute notebook.ipynb --kernel_name=databricks` がそのままパイプラインで動く
- **AI ツール自律実行**: Claude Code / Codex CLI がノートブックを実行・検証・修正できる
- **ローカルと本番で同じコード**: Databricks クラスター上で動くので、環境差異がない

ローカル開発環境が整ってはじめて、テスト・CI・AI の全ての品質管理ツールが使える。

2.11. 他プラットフォームで真似する人へ

必要な要素は3つ:

要素	役割	例
コード実行 API	コードを文字列で渡してリモートで動かす	Command Execution API 相当
ファイル同期 API	ローカルファイルをリモートに送る（ディレクトリ構造を保持）	Workspace Files API 相当
セッション管理	REPL を stateful に保つ（変数が次のセルでも生きる）	contexts/create 相当

この3つが揃っていれば `do_execute` を override すると完全リモート実行カーネルが作れる！

3. エコシステム

3.1. 開発品質を支えるエコシステム

パターン / ツール	役割	ポイント
Notebook は関数実行のみ / ロジックは `.py`	テスタビリティ	`.py` に切り出し → `pytest` で単体テスト
uv	依存関係管理	`uv sync --active` で Databricks 環境に直接インストール
mise + pre-commit + Renovate	ガードレール	コミット時 Lint / 依存更新の自動化

4. まとめ

4.1. まとめ

やったこと

課題	解決
ローカル開発との乖離	jupyter-databricks-kernel で完全リモート実行
AIツールからの自律実行	jupyter-databricks-kernel でClaude Codeが実行
コード品質・テスタビリティ	`.py` 切り出し + uv + ガードレール

jupyter-databricks-kernel がローカル開発を可能にし、堅牢な開発体制を整備することができるようになった！